

S) labels: Indicates that the issue is related to usability problems surfaced in one of our SUS research efforts. More specifically, issues related to SUS that are prioritized can be labeled with the corresponding Fiscal Year and Quarter. For example: SUS::FY22 Q2 - Incomplete. Learn more about SUS score as a UX Department Performance Indicator

- Regression label: Indicates a bug introduced in the latest release that broke correct behavior (see the contribution guidelines for more info).
- UX scorecard label: Indicates the primary issue or epic for the UX Scorecard. We use this label to help us easily find current work and track efforts over time.
- UX scorecard-rec label: Indicates this issue is a recommendation that was a result of a UX scorecard review. It's OK if the issue was created prior to the scorecard being done; it can still be pulled into the set of recommendations.
- CM scorecard label: Indicates the primary issue or epic for the CM Scorecard. It is used to easily find current work and track efforts.
- cm-scorecard-rec label: Indicates this issue is a recommendation that was a result of a CM Scorecard.
- Actionable Insights document learnings from research that need to be acted on.
 - Actionable Insight::Exploration needed: A research insight derived from a UX research study that requires further exploration.
 - Actionable Insight::Product change: A research insight derived from a UX research study and requires a change to the product experience.
- Type labels: Used to track feature, maintenance, and bug issues and MRs. UX Leadership are active participants in influencing the prioritization of all three work types. See also who are the DRIs for prioritization.
- Theme labels can be created to group issues that solve a similar user experience problem but don't have a category. This can be especially useful for a user experience that spans the product. These issues still require a UX label.
- UX: Feature Discovery Improvement: Indicates issue may improve feature discoverability.
- UX: Onboarding Improvement: Indicates issue is a potential onboarding improvement.

UX Calendar

The UX Calendar (internal only) is the SSOT for our team meetings. You can find the details for UX calls, UX Forum, and other team meetings here. These meetings are open to everyone in GitLab. Anyone in the UX department can add events to the Google Calendar. Managers and above can make changes and manage sharing, while ICs can make changes to events. Please reach out in the uxleadership Slack channel with any questions or requests.

UX All Hands

The UX All Hands meeting takes place every six weeks, with two sessions to accommodate both EMEA/AMER and APAC/AMER-friendly time zones.

The purpose is to share company updates, stay connected, and receive feedback.

The entire UX department is invited, though anyone at GitLab is welcome to attend and contribute to the All Hands agenda (internal). As with all general meetings at GitLab, attendance is optional though encouraged, and will be recorded.

Retrospectives

To understand the specific challenges faced by the UX Department, we hold an async UX retrospective after every milestone. This retro is carried out through a new Issue created for the recent release in the ux-retrospectives project. The goal is to evaluate what went well, what didn't go well, and how we can improve.

UX Forum

{{% product/ux-forum %}}

UX Week in Review

The UX Week in Review is an asynchronous document that includes important updates for everyone in the UX department. This doc is editable by anyone at GitLab and everyone can contribute.

Reminders are sent out weekly in the ux Slack channel to add and read updates. UX Managers will receive a monthly reminder in the uxleadership Slack channel to add project updates from across their team.

UX Talent Assessment

In UX, we utilize performance factor worksheets (· internal only) as a way to facilitate talent assessment and growth conversations between manager and their direct reports. These worksheets are available in Google Sheets format and the spreadsheets include tabs for a mid-year and year-end review, as well as a tab to list Achievement, Strengths, and Opportunities throughout the year.

It is strongly encouraged for each team member to have their own worksheet created at the start of the fiscal year so that it can be used as a tool throughout the entire year.

Performance factor worksheets can be utilized to help efficiently complete the year-end company talent assessment program within Workday.

Meet some of our team members

- Valerie Karnes - Director of Product Design
- Jacki Bauer - Product Design Manager
- Justin Mandell - Product Design Manager
- Taurie Davis - Senior Product Design Manager
- Rayana Verissimo - Product Design Manager
- Jeremy Elder - Staff Product Designer, Foundations
- Austin Regnery - Senior Product Designer
- Anne Lasch - Senior UX Researcher
- Emily Bauman - Senior Product Designer
- Will Leidheiser - Staff UX Researcher

SECTION: product/ux/collaborators-playbook.md

What's it for?

This page acts as a quick reference of mental models for sync and async collaboration. Its purpose is to create a common language for collaboration between designers, GitLab team members and the broader GitLab community. It should be used to understand and label patterns in collaboration, pain points, where we get stuck and how to get unstuck. It is not our design workflow.

Why is this important?

Collaboration is one of GitLab's key values. At GitLab, we believe that everyone is a designer and everyone can contribute. This means the UX department is a humble facilitator of design. The design team can support collaboration across GitLab (and our broader community) by leveraging and sharing the mindsets, methods and tools of design thinking.

Mental models

2 steps forward, 1 step back design

At GitLab we ship using the Minimal Valuable Change (MVC). Designing in this context can often be confusing for newcomers because it's important to consider the big picture as well as the steps for how we get there. To deal with this dichotomy, we design by initially thinking about the mid to long-term vision and then reducing the scope of the experience (in a lovable way) to make implementation quicker. In other words, we design by taking 2 steps forward and 1 step back.

Actions

- Use this model with your team to break a vision down into a series of small (MVC) issues

Divergent & convergent thinking

From a young age we are often trained to jump to solutions as quickly as possible. This prevents us from taking the time to explore fully all our options. We can overcome this challenge by understanding and leveraging divergent and convergent thinking. These are two of the foundational modes of creative problem-solving:

- Divergent thinking (go broad): Generating lots of different ideas and exploring options without too many constraints in place.
- Convergent thinking (focus in): Reflecting on your options and ideas, combining them together in unique ways, analysing them, and converging on a solution (with constraints in mind).

Actions

- Watch short video
- Discuss with your team whether it is time for diverging or converging
- Leverage a synchronous collaborative cycle
- Summarise text in a divergent thread and ask to converge on a solution (example)

Ladder of abstraction

We can't design in isolation. All the decisions we make are in the context of a broader system. We must zoom in and out between the system and its component parts to do our best work. Unfortunately, humans can only hold 5 (plus/minus 2) pieces of information in working memory at any one time (see cognitive load, Miller's Law). Therefore, we need to think at different levels of abstraction to manage this limitation. The ladder of abstraction is a useful mental model to recognise what level of abstraction you are currently working in. You can move up the ladder (more abstract) by asking "why?" or "what does that mean?". You can move down the ladder by asking "how does that work?" or "can you give me an example?".

Example:

- Very abstract: "This is how I commute to work"
- Moderately abstract: "This is my personal transportation device"
- Concrete: "This is a bicycle"

Actions

- Read "Up and Down the Ladder of Abstraction"
- Use the "5 Whys exercise" to move up the ladder of abstraction
- Use sketches and prototypes to move down the ladder of abstraction

Resources

- Salesforce Workdifferently
- Salesforce Workdifferently: Method cards
- IBM enterprise design thinking

Collaborative exercises

As designers, we're generally embedded within stage groups. As we work to create designs, it can be helpful to include our engineering and product counterparts in our ideation processes. Collaborative exercises can sometimes be quite useful for this purpose. Conducting these exercises periodically within our teams can help build trust and generate buy-in. It can also help surface ideas and things to consider earlier rather than later in the design process.

As we are distributed globally you may opt for either an async or a sync collaborative exercise, depending on your goals and desired outcomes.

Below are a few of the exercises teams have run, organized by which stage of the design process you're in.

Product direction and vision

When you're defining your product direction and vision, putting together an async board for your team to review your ideas · and add their own · can be quite useful. Some teams have run product direction workshops, which can either be broadly or more narrowly focused, depending on your needs.

An altitude exercise is another approach. An altitude exercise solicits feedback at the satellite-level, airplane-level, and ground-level. This exercise can help gather feedback about strategy and the particular design details simultaneously.

Ideation

If you're at the ideation stage of the design process, you might consider an asynchronous ideation workshop. If you have a number of options you're weighing as a team, a sync lightning decision jam could help the team align on a way forward.

Specific design feedback

If you've developed a design direction and want to give the team space to ask questions synchronously, an "ask me anything" style meeting can be useful. When running this type of session, ensure you share your design direction prior to the meeting so everyone has a chance to review and you can gather the best feedback.

Alternately, if you'd like feedback on common areas of confusion or on the technical feasibility of your proposal, putting together an async board for your team to review can be a great way to start these conversations.

SECTION: product/ux/competitor-evaluations.md

What are competitor evaluations?

Competitor evaluations help us understand how a competing product addresses the Jobs to be Done (JTBD) that our product also tries to address. It evaluates solutions.

These differ from a competitor analysis, which compares business models and strategic positioning in the market (think SWOT analysis). A competitor analysis evaluates companies.

In some cases, a competing company may solve a narrower aspect of DevSecOps than GitLab. In contrast, another company might have a portfolio of solutions that add up to the full breadth of GitLab. Understanding the strengths and weaknesses of competitor offerings can help us identify gaps in our own functionalities and also inspire us to come up with better solutions.

Selecting competitors to evaluate

At GitLab, we rely on the JTBD framework to understand the goals that people want to accomplish using our product. Through these jobs we identify other competing products in the market that are helping users accomplish similar goals. When deciding on the level of the job that you want to work with for this exercise, keep in mind the scope of the JTBD comparison. Consider also the size and level of different products when conducting a competitor evaluation.

Direct and indirect competitors

GitLab's direct competitors are the ones that offer similar solutions within the DevOps industry and targets the same audience. Generally speaking, these competitors are the first

ones that come to mind when talking about different solutions, or the ones customers compare us with in the market. You can find some examples of our direct competitors in the Gartner website.

Indirect competitors are products or services that help users perform a similar job through a different solution that doesn't necessarily address the DevOps market. We look beyond direct competitors to understand how to innovate.

Technology is a fast-evolving landscape. To avoid any surprises in the future, it is important that we consider both indirect and direct competitors when conducting an evaluation.

Objective of performing a competitor evaluation

There are several objectives for conducting competitor evaluations:

- Highlight opportunities and make informed recommendations to improve products and place GitLab ahead of the competition.
- Speed up the design process by making less risky assumptions and increasing our confidence in proposals based on the insights found.
- Avoid the same pitfalls that our competitors face.

To help determine the maturity of a category across industry more accurately, our team is exploring a Competitor Comparison process that fits into the existing Category Maturity Scorecard process.

How to get started

Start by creating a new issue using the competitor evaluation issue template. The issue template includes a list of steps, as well as helpful links on how to go about completing an evaluation.

Legal considerations

Follow the instructions on the Legal Product Guidance internal handbook page when planning and conducting a competitor evaluation.

Learn more about performing competitor evaluations

SECTION: product/ux/design-sprint.md

Purpose

The purpose of a Remote Design Sprint (RDS) is to quickly create a shared understanding of a complex problem and solutions to move forward while collaborating in a remote environment.

It has the same benefits as non-remote Design Sprints while incorporating techniques of synchronous and asynchronous activities.

The RDS process is based on Google's Design Sprint methodology, and adjusted using AJ&Smart's Remote Design Sprint 2.0 in accordance with Jake Knapp, the creator of the original Design Sprint.

When to opt for a Remote Design Sprint

You should use the RDS when you need to quickly find a design solution or direction for a large problem. Sometimes, these problems span multiple stages or groups.

How to pitch a Remote Design Sprint

When making a request to conduct a RDS, work closely with your Product Design Manager to gather information needed to help prioritize the effort with Product Management. Be clear about the scope, duration, and level of effort needed from RDS participants and include how the RDS will get results faster for upcoming milestones.

Who should participate in a Remote Design Sprint

You should keep the RDS team to around 5-7 people. More than that becomes difficult for the facilitator to manage and less than that can limit the diversity of backgrounds and knowledge needed to solve the problem. RDS participants should be a mix of counterparts from each participating team, such as UX (Product Designers, Researchers, Tech Writers), Development, Product Management, Sales, Customer Support. The more diverse the better, but be sure they're relevant to the challenges being worked on and that they have the time to contribute effectively.

Remote Design Sprint methodology

Your team's availability and the priority of the problem needing to be solved will determine how compressed your RDS may be. Synchronous meetings can allow you to complete exercises more quickly. However, it is possible to complete many of them asynchronously.

RDSs work best with a mix of asynchronous and synchronous sessions. This gives your team ample time to plan, participate, and complete any asynchronous homework prior to synchronous workshop sessions while balancing other daily tasks. Review the RDS Issue Template for suggestions on which exercises work better synchronously or asynchronously when considering your RDS timing.

We recommend keeping your RDS to two weeks or less. Any more than that and you may suffer from loss of momentum. Regardless of the actual duration, the steps, methodology, and exercises are essentially the same. However, you may find that some of the exercises are unnecessary to complete as part of your RDS because you already have the content or data sorted out. In cases like this, you should feel comfortable removing those exercises from the Figjam template you copied during your setup outlined below. Note: Before you remove an exercise, be sure to read through the details and instructions to ensure that you truly do not need it.

Each day of the RDS may take more than 24 hours to allow everyone, across time zones, the same time to complete asynchronous activities (especially if there are large gaps between the earliest and latest timezones). This is okay. If participants have a 12+ hour gap or more, it is best to plan each day of the Sprint to take 48 hours so everyone has a full work day.

Setup

1. Create an Epic using this template which will be the SSOT for your RDS's goals and process.
 1. Be sure to read through it completely and thoroughly as it outlines the details of each exercise.
 1. Gather all of the supporting research data you have, linking any Insights to this RDS Epic.
 1. Create issues for each day of the Sprint using the following templates and link them to the main epic.
 1. Optionally - create a planning issue and link it to the epic. This can be used to help plan the problem, Sprint team, time zones and more.
1. Work with your Product Manager to form your team of RDS participants (5-7 RDS participants with diverse backgrounds and knowledge of the problem space is recommended).
1. Create a new folder in Google Drive that can be shared with your RDS participants. This will act as a repository for any assets created during the asynchronous homework exercises.
1. Create a duplicate of the Figjam Remote Design Sprint template that will be used as the primary working area.
1. Create a special Slack channel that is specific to this RDS

Once the above is done, you'll need to decide how to break up your sessions. This is dependent on how long you plan your RDS to last (4 days, spread over 2 weeks, or somewhere in between?). If you're able to do the standard RDS over 4 days then you can use the Figjam template as a guide. Otherwise, you'll need to get creative and balance your team's availability with the needs of each session's included exercises and how they work together.

Next, you'll need to onboard your participants. Start by walking your Product Manager through what a RDS is if you haven't already. Inform them that they will play the role of the "Decider" and explain that the Decider is responsible for making the final decisions on which challenges or concept(s) should progress to the next step, etc. This should be done separately in a sync call or over a Slack private discussion to ensure they fully understand their role and to answer any questions they may have prior to notifying the rest of the team.

Once this is complete, add a new comment and CC each of the participants in the RDS Epic you created during your setup, inviting them to the RDS and requesting that they read through the Epic thoroughly. Also, let them know that you will be reaching out soon either via a survey or through synchronous discussions so you can get an idea of what their pain points/challenges are around the RDS topic. In the Design Sprint world, this is known as The Expert Interview where you as the Facilitator want to discover and understand the challenges the team wants to try and solve about the RDS topic (See the Issue template notes for more details).

After notifying your RDS team within the Epic:

- Share the Google Drive folder with them
- Add them to the Slack channel you've created
- Provide a link to the RDS's Figjam board.

This should all be done at least 1 week before the RDS is scheduled to begin.

Balancing asynchronous and synchronous collaboration

While it's possible to participate in all of the RDS activities completely asynchronously, synchronous collaboration is very helpful throughout the RDS. Use a timezone tool to better understand what time zones you are working with and they may or may not overlap. When reaching out to sprint participants who fall out of the facilitator's working hours, discuss with them how they prefer to participate... async as possible, or perhaps they're open for working slightly different hours during the sprint week.

Each day of the RDS may take more than 24 hours to allow everyone, across time zones, the same time to complete asynchronous activities (especially if there are large gaps between the earliest and latest timezones). This is okay. If participants have a 12+ hour gap or more, it is best to plan each day of the Sprint to take 48 hours so everyone has a full work day.

General guidelines for facilitators

The Facilitator is responsible for the overall organization of the RDS, planning, and communication. They have by far the largest amount of responsibility in a RDS.

- They set the rules and pace of the workshop and ensure everyone understands what is required during each exercise.
- During sync sessions, the Facilitator is responsible for explaining each exercise and engaging with the RDS participants.
- They should also select someone to be a co-facilitator to assist with logistics, such as your Product Design Manager. The co-facilitator can assist during sync sessions keeping everyone on task or answering questions in chat as the Facilitator is facilitating. They can assist in answering questions if the Facilitator is in a different time zone.
- A faccilitator should be aware of how time zones affect asynchronous activities, and ensure all communication is given before the start of the earliest timezone. This allows everyone in the Sprint to have the same amount of time to complete activities.

General guidelines for RDS participants

RDS participants are responsible for participating in the RDS activities as outlined by the Facilitator.

- It's recommended to attend the sync sessions. During sync sessions, follow the general guidelines for effective all-remote meetings.
- If the sync sessions are out of the participant's working hours, it's important to participate asynchronously, for example by sharing a video recording about the RDS deliverable for the day or any questions the RDS participant might have.
- All asynchronous tasks must be completed before or on the due dates. If there is a problem achieving any of the above this must be communicated to the Facilitator or co-facilitator as quickly as possible so adjustments or assistance can be provided.
- Honor the Facilitator's directions. They're the guide for the entire process.
- The Decider has the final, ultimate decision at the end of each activity.

Tools

Here is the list of tools for RDS preparation, collaboration, and documentation. Prior to the RDS make sure you have access to all of the following:

- GitLab
 - Each RDS day's outcomes and materials will be documented in a separate issue under the RDS epic.
- Figjam
 - RDS participants can join as anonymous but there is a need to be able to identify input against names, so creating an account beforehand is recommended.
 - Figjam will be used for most of the RDS collaboration. Some of the things done in Figjam:
 - Create artifacts like affinity diagrams from RDS participants' input
 - Use post-its to comment on each other's points and to add notes
 - Vote on ideas and solutions
 - Create the first draft of the prototype.
 - The Figjam link to the collaboration project will be provided in the issue before the start of the RDS.
- Video and/or screen recording tool (Loom, QuickTime, Zoom, etc.).
 - As part of the pre-RDS homework, RDS participants will be asked to record a short Lightning Walkthrough video. RDS participants can use any tool they feel comfortable with as long as it can capture their screen, mouse pointer, and audio.
 - The facilitator will also provide walkthroughs of each asynchronous activity, which participants will need to watch.
- A4/Letter-sized paper (preferably white blank), Sharpies/Pens (please don't use a pencil because it doesn't create enough contrast for photos).
 - Parts of the RDS will involve some (async) ideation via sketching, RDS participants will need a writing utensil (Sharpies are preferred because they force you to draw at a lower fidelity because small details aren't necessary at this point) and some paper. This is the most fun part of the RDS where RDS participants get into a design thinking mindset and can appeal to their creative selves. Don't worry, it's not about artistry, it's about ideas and collaboration.
- Camera (phone or other) or scanner
 - RDS participants will need to upload sketches as images for the facilitator to prepare the material before the next sync meeting. RDS participants can take a photo with their phone or use a scanner if available.
- Post-it notes (Optional)
 - If you enjoy taking notes using post-it notes make sure to have some of them available as well. The upside is that they will make you feel more like you are in a workshop and will help the ideas flow. The downside is that you will have to digitalize the ones you want to share with the team in Figjam.

Past Remote Design Sprints

List of past RDSs:

- Auto DevOps Design Sprint (Figjam board)
- Configure Stage - Design Sprint (Kubernetes management focus)] ([Figjam board)
- Growth · User's dashboard design sprint (Figjam board)

SECTION: product/ux/heuristics.md

Intro

UX Heuristics are guidelines we can use to grade experiences in our product. They can also be useful while designing because you can use them to make design decisions or identify parts of your design that still need work.

This list can change and the handbook version is the SSOT. It's based on best practices in the UX field, as well as feedback from our customers as to what they want their GitLab experience to be like.

Templates

- Competitor Evaluation
- UX Scorecard Issue Template

Heuristics

| Category | Heuristic | Description |

| ----- | ----- | ----- |

| Usability | Visibility of system status | Communicating the current state allows users to feel in control of the system, understand available actions and how to use them to reach their goal, and ultimately establishes trust in the brand. This heuristic differs from the "real-time user interface" heuristic (see below) in that it is a deeper understanding of what is going on with the tool or product (consider a dashboard), whereas real-time user interface is communicating something about the tool or product when change occurs (being notified immediately if there is a dramatic change to one or more metrics on that dashboard). (source: NN/g) |

| Usability | Flexibility and efficiency of use | Shortcuts (unseen by the novice user) speed up the interaction for the experts such that the system can cater to inexperienced and experienced users. Frequent actions can be customized or automated. Users can customize the tool for their needs and personalization is used appropriately to reduce tedious tasks. (source: NN/g) |

| Usability | User control and freedom | Allow people to exit a flow or undo their last action and go back to the system's previous state. (source: NN/g) |

| Usability | Recognize, diagnose, recover from errors | Make error messages visible, reduce the work required to fix the problem, and educate users along the way. (source: NN/g) |

| Desired Characteristics | Real-time user interface | When a change of system status occurs, the interface instantly provides feedback to users. (see: The future of MRs: Real-time collaboration) |

| Desired Characteristics | A tool the whole team can use | GitLab should be usable regardless of your role in DevOps or your technical ability. (see: GitLab Direction personas) |

| Desired Characteristics | Minimal setup required | Configuration choices that are well thought out and based on current best practices. (see: GitLab Product Principles convention-over-configuration) |

| Desired Characteristics | Documentation is easy to use | Documentation is clear, concise, well organized, easy to follow, easy to understand, and technically accurate. |

| Onboarding | Features communicate the problem being solved and value to a new user | When users explore an unfamiliar feature, they should be able to quickly understand how it is or isn't relevant to them. |

| Onboarding | Features/workflows have clear calls to action. | A call to action should make it clear what is to be done to move the task forward. At the very least, there should be links to documentation for assistance. At best, guided setup. |

| Onboarding | Tasks are easy to learn, or if highly complex, have setup support in the form of in-app guidance, defaults, templates or wizards. | Tasks should be intuitive first and foremost, and when they are complex, they must provide tools to enable learning. |

AI Heuristics (for probabilistic features)

Credit to Emily Campbell at the Shape of AI

| Category | Heuristic | Description |

| ----- | ----- | ----- |

| Usability | Purposeful and Needful | The AI solves for a real and significant need in a meaningful way, and makes sense within its surrounding context. Users easily understand the capabilities available. |

| Usability | Input Clarity and Ease | Users understand how to initiate an interaction and can communicate their intent. |

| Usability | Context | Users can understand what context and information the AI has available to it. Users can add additional context that fits the task. |

| Performance | Result Quality | Results are accurate, relevant and helpful. Output is readable and visually appealing. |

| Transparency | Transparency of Results | Users can understand why they got the answer they did. Sources are identified and linked as appropriate. |

| Usability | Customization and Tunability | Users can fine-tune their inputs to easily generate outputs that match their specific needs and expectations. |

| Performance | Branching and Recall | The AI maintains conversational context throughout or across interactions, allowing users to explore different paths and easily return to the main thread. |

| Usability | User Autonomy and Control | Users maintain control over the AI through mechanisms that let them guide, direct, and control the interaction. When the interaction is complete, users can use the results elsewhere. |

| Transparency | Logical Transparency | The AI clearly communicates its decision-making processes, improving user comprehension and trust. |

| Transparency | Ethical Integrity and Trustworthiness | The AI adheres to ethical standards, minimizes bias, protects privacy, and ensures transparency to foster user trust. Risks to the user are managed appropriately. |

| Transparency | Identification and Honesty | Users can distinguish AI inputs and outputs from human-generated content. The AI is honest and transparent about its capabilities and constraints. |

| Performance | Continuous Learning | The AI continuously improves, learning from user feedback and data to enhance its functionality. |

Scoring

Scoring these heuristics can have an element of subjectivity, but the more you involve other users in the evaluation the less subjective it will be. Heuristic scoring is optional, but consists of scoring all 11 heuristics for a part of the product on a 1-5 scale using the scoring table below. This scoring sheet allows you to easily record your responses and determine an overall score (average of individual scores for the 11 heuristics).

| Badge | Score | Summary | Description |

| --- | --- | --- | --- |

A	5	Excellent	The heuristic is met in every way and optimized. Future improvements are minor. Users may even report that completing the task is delightful. Users report exceeded expectations and high confidence of task success.
B	4	Good	The heuristic is met in almost every way, with no more than minor exceptions. The task is streamlined, pleasant and highly usable. Users may report exceeded expectations and medium high - high confidence in task success.
C	3	Average	The heuristic is mostly met. Exceptions could require minor workarounds, but the completing the task is relatively straight forward and overall meets expectations. Users may report only moderate confidence in task success.
D	2	Below Average	The heuristic is lacking in significant ways leading to frustration. The task requires workarounds to complete or otherwise does not meet expectations. Users report low confidence in task success.
F	1	Bad	The heuristic isn't met. It may be completely ignored, or so poorly implemented that the task is impossible to complete.
-	-	Unknown	This task hasn't been scored.

If you're completing a UX Scorecard, please use the UX Scorecard Grading Rubric.

References

- Nielssen Norman Group
- UI Tenets & Traps

SECTION: product/ux/thinkbig.md

Think Big

Purpose

A Think Big is designed to create a shared understanding of purpose and goals with all members of a stage group. Everyone in the stage group is invited to participate in a facilitated discussion around vision, roadmap, user research, design, and delivery of upcoming product features.

Think Big's are meetings that drive knowledge-sharing across the organization at the stage group and section levels. Participants are given the opportunity to review and ensure short-term activities are aligned with long-term goals.

Watch past recordings in the GitLab Unfiltered YouTube channel.

Cadence

Think Big meetings occur once a milestone (once a month).

When to opt for a Think Big

Having a synchronous meeting with a large number of individuals may feel like it goes against GitLab's Values and culture. It is essential to evaluate the Think Big meetings regularly to

ensure they are valuable to the team. Some questions to ask could include:

- Are the topics designed and delivered in a way that brings the team together?
- Are the discussions harmonizing the group on the long-term strategy of the stage or group?
- Having an entire team on a call for an hour can be considered expensive. Are the sessions a good use of everyone's time?
- Is everyone included in the conversation? Do all the participants in the session have an opportunity to discuss ideas or provide feedback?
- Do the conversations evolve and consider feedback from the team to make the session optimal for its goal?
- Are the goals and outcomes of the sync discussion available to everyone after the session?

If any of the above questions are answered with a "No" it is a sign to reevaluate the topics, discussions, and structure of Think Big and iterate to ensure the team gets the most value out of the meeting as possible.

General guidelines for facilitators

- Think Big meetings should be conducted in groups of no more than 10-15 (to encourage participation).
- The agenda template can be found by searching for Think Big in Google Drive.
- Divide the meeting into modules with each module averaging 15 minutes. A full session can typically cover three topics.
- As the meeting starts, look for an ordered list of discussion topics in the agenda doc. This will be the order for participants to go in.
- There are many ways to gather feedback in a synchronous meeting. An example is to use a round-robin, turn-based structure that gives everyone on the call an opportunity to share their thoughts quickly and efficiently.
 - The facilitator (usually a Product Designer) will kick off the meeting by quickly reviewing the guidelines and starting the 15-minute timer.
 - The facilitator presents the problem or design by communicating what type of feedback they need. For more genuine reactions and feedback, keep the explanation as short as possible.
 - Following the order in the agenda, participants take turns asking relevant questions and providing a single piece of feedback to the design. Each "turn" should be limited to about 1 minute.
 - Repeat this turn-based process until time runs out or all the participants "pass".
 - If you need inspiration for feedback, consider taking a few different hats for a spin!
- Action items from the meeting will be turned into issues with a think big label, such as Package:ThinkBIG!.

General guidelines for presenters

- Add the discussion items to the think big agenda doc ahead of the meeting and make sure you link to relevant issues, research insights, and prototypes.
- There are many ways to gather feedback in a sync-meeting, one example is to use a Round Robin turn-based structure that gives everyone on the call an opportunity to share their thoughts quickly and efficiently.
 - Try to be quick, as each turn should only last around 1 minute. During your turn, you can do a few things (in order of priority):
 - Ask questions to the facilitator.

- +1 or -1 someone else's feedback.
- Provide one(1) piece of feedback.
- "Pass" - you can skip your turn.
- You officially end your turn by calling out the name of who is next.

Suggested discussion items

Research insights review

After completing Problem or Solution Validation, Product Managers and Product Designers walk away with robust data about our customers and their needs. The Think Big meeting is a perfect forum for sharing the plan, process, and results of those research efforts.

Upcoming research discussions involve the entire team and give everyone involved in creating a product or feature an opportunity to raise questions to ask the users.

Reviewing past research initiatives and discovered UX-Research Insights with the whole team helps everyone understand what our users need and how the features being built will help them.

Epic overview

Reviewing a new epic with the whole team helps everyone understand the large scale vision and the logical breakdown formed. Having this conversation opens opportunities for the team to collaborate on how to break down large work into MVC deliverables in a logical and efficient order.

Feedback during a Think Big

- Feedback should revolve around the areas the presenter has pointed out.
- Feedback can be either positive, negative, or neutral. Helping a presenter know what is working is as important as what could be improved.
- One piece of feedback can build off of another person's feedback.

Remember, the goal is to capture a quantity of specific feedback. While it may be tempting to start discussions around the design choices and feedback, this activity doesn't make for a proper forum. The designer will follow up with reviews asynchronously afterward in the issue(s) to start discussions and conversations around the feedback.

Think Small

A think big meeting may be accompanied by a think small meeting. While the think big focuses on divergent thinking about a problem and the solution space, the think small portion focuses on convergent thinking to help define the smallest step that the team can take today.

At GitLab, a great outcome of a Think Small meeting is having a small issue that solves a customer problem and gets them closer to the larger outcome discussed in the Think Big.

General guidelines for facilitators

- Start the meeting by doing a recap from Think Big:

- Review the User Personas you have discussed during the Think Big. Shortly revisit their needs and challenges.
- Revisit some ideas for solutions that you have previously discussed.
- Review design files that may already exist for the solution.
- Think about and discuss what could the team do in the next milestone to test these ideas.
- Discuss what can be the hardest part of the problem to solve in order to get ready for the possible roadblocks
- Define and agree on the steps to take.

SECTION: product/ux/user-journeys.md

User journey mapping at GitLab provides a strategic framework for teams to understand the complete journey of customers and GitLab users as they discover, evaluate, adopt, and integrate our DevSecOps platform. By systematically mapping these experiences:

- We identify friction points, opportunities for education, and moments that matter most to users.
- We create seamless pathways that increase first orders, accelerate successful onboarding, and strengthen long-term retention across our customer base.

Key terminology

- User journey: A user journey is the complete sequence of experiences, interactions, and touchpoints that a user has with GitLab over time to accomplish specific goals or outcomes. It encompasses the user's progression through various stages and touchpoints relevant to their goals across different stages of the software development lifecycle. The specific details captured (such as emotions, pain points, or granular steps) will vary based on the elevation level of the journey map being created.
- Golden journey: A golden journey is the most valuable and strategically important user path through GitLab's platform that maximizes both user success and business value. It represents the ideal, high-frequency route that users take to achieve core outcomes while driving key business metrics such as activation, retention, expansion, and monetization. Golden journeys typically span multiple stages of the software development lifecycle and serve as the foundation for product strategy and cross-functional alignment.
- Journey map: A journey map is a visual representation and documentation of a user journey that captures the user's workflow, stages, and key opportunities across different touchpoints. The specific details captured (such as emotions, pain points, or sub-steps) will vary based on the elevation level of the journey map. It serves as a shared artifact for cross-functional teams to understand, analyze, and improve the user experience.

Journey map elevations

Journey map elevation refers to the different levels of detail and scope at which user journeys can be mapped and analyzed, each serving different strategic and tactical purposes within the organization.

Journey map elevation levels connect and inform each other, with micro-level insights about specific interactions feeding into mid-level understanding of complete workflows, and mid-level findings informing macro-level strategic decisions.

Macro level

- Purpose: Strategic alignment and ecosystem understanding
- Timeframe: Covers months to years capturing the full customer lifecycle from initial awareness through long-term use/advocacy
- Scope:
 - Focus on high-level phases, major milestones, and business outcomes
 - Show cross-service and cross-channel experiences
 - Include long-term emotional arc and loyalty drivers
 - Identify ecosystem-wide improvement opportunities
- Artifacts: TBD

Mid level

- Purpose: Cross-functional alignment and end-to-end experience optimization
- Timeframe: Covers days to weeks for a specific user goal or service experience
- Scope:
 - Focus on specific user scenario from start to finish
 - Include detailed pain points and emotional journey
 - Show specific touchpoints and channels
 - Identify tactical improvement opportunities
- Artifacts: TBD

Micro level

- Purpose: Interaction optimization and usability improvement
- Timeframe: Covers minutes to hours for specific task completion
- Scope:
 - Focus on granular user flows and micro-interactions
 - Include detailed interface elements and user actions
 - Show moment-by-moment emotions and friction points
 - Identify specific UI/UX improvements and metrics
- Artifacts: TBD

SECTION: product/ux/ux-forum.md

About UX Forum

{{% product/ux-forum %}}

Purpose

- Increase awareness of the value delivered through UX by highlighting business, product, and user outcomes

- Increase visibility into findings, opportunities, solutions
- Increase exposure of UX activities to the broader organization
- Increase work quality by promoting cross-stage collaboration in the meeting and after it

Audience

- Product designer, researcher, and/or technical writer (speaker)
- UX department
- Product division
- Engineering division
- Relevant leadership (VP of UX, VP of Development, VP of Product, E-Group)

Schedule

Forums are scheduled every two weeks. They are 60 minutes long to allow up to three team members to share for 15 minutes, plus 5 minutes of discussion time with the attendees.

- To be inclusive of our distributed team, we alternate the meeting start time every other occurrence to be either Americas/LAC/EMEA friendly or APAC friendly.
- Each forum is hosted by a Product Design Manager.
 - Due to time zone restrictions for managers, the APAC UX Forum is hosted by another Product Designer, Technical Writer or UX researcher.
- Product Designers are assigned dates based on a randomly generated rotation:
 - We ensure each designer has the opportunity to share twice a year.
 - Gaps are left throughout the schedule for additional speakers.
- Anyone can sign up to share and discuss their work. Coordinate with the host to claim an available slot.
- Each person who shares is expected to fill out the agenda (attached to the meeting invite) for that week prior to speaking.
- It is the responsibility of the host to know when they are scheduled to host and the responsibility of the speaker to know when they are scheduled to share. To help, a Slack reminder has been set up in the ux channel to review the schedule.

If a speaker can't share on their assigned date, it's their responsibility to make a trade. To make a trade:

- Announce in the ux channel and during the UX weekly call that you'd like to trade.
- After you identify a trade, make an MR to update the schedule.
- Ask your Product Design Manager to review and merge.

```
{{% include "includes/ux-forum-schedule.md" %}}
```

```
<a href="https://gitlab.com/-/ide/project/gitlab-com/content-sites/handbook/edit/main/-/assets/includes/ux-forum-schedule.md" class="btn btn-primary">Update schedule</a>
```

Sharing your work

You have up to 15 minutes to share some of your work.

If you need more time to share and discuss a complex topic, ask the host about having fewer

people speak at that forum.

Preparation for a forum should be minimal.

You should come prepared with just enough to tell a story and elicit feedback from the audience.

Successful forums share these aspects:

- Informal:

Avoid creating dedicated slides or assets, but use your best judgment to determine what's most effective: a prototype, a series of mockups, a process diagram, a journey map, a few slides, or something else.

The UX Forum is to share and discuss your work informally, not fancy storytelling.

- Just enough sophistication:

Help us strive for a low level of shame and share your work with as little sophistication as possible.

While some topics gain from a more sophisticated story, this sophistication raises the perception of quality, puts unnecessary pressure on other speakers, and might make it harder for others to engage.

- Relatable:

Structure your story with the audience in mind, to elicit feedback from them, and highlight potential overlaps.

This makes it easier for people to relate, find value, and engage.

For example, intentionally pinpoint overlaps or add questions for the audience.

- Iteration mindset:

Use it as an opportunity to learn from and collaborate with others.

Make iteration shine not only in the work you've done so far, but how you might iterate forward and what's missing.

For example, list your current challenges, open questions, and where others might help.

See helpful tips.

Examples of minimal forums:

- Fulfillment: Improve provisioning and onboarding of new sales assisted GitLab com subscribers
- Secure: Validating and prioritizing your roadmap
- Secure: Group-level security scanner status widget
- Secure: Security Workflow Iterations

Preparation

- Prior to the UX Forum, add your specific topic to the meeting agenda.
- If this is an APAC forum and there is no host yet, consider hosting.
- Provide context for the problem:
 - The scope of the problem
 - Why was/is it important to solve?
 - What did we learn during research?
 - Any constraints that impact the solution?
- State the desired goals of the work:
 - What is the desired business and customer outcome?
 - Ideally, describe the JTDB

- When, [user's context]. I want to [user's goal]. So I can [user's desired outcome].
- What were/are the constraints?
- How did you/are you planning to iterate toward an MVC?
- Walk your audience through the solution iterations:
 - Be yourself and tell the story of the work.
 - Use existing mockups and flows, rather than creating something new.
- Invite discussion:
 - How might this overlap with or help other teams?
 - What are the current challenges, next steps, and open questions?
 - How might others approach this or help you?

Helpful tips for those sharing

- Introduce yourself and provide context for your topic.
- Make the participant experience an enjoyable one: avoid moving around too fast (for example, when showing a Figma prototype), or back and forth between views because you forgot to say one thing at the previous one.
- When sharing your screen, consider going fullscreen so that the audience can see the details.
 - Useful Figma keyboard shortcut on macOS to show/hide UI: `⌘\` or `⌘.`
- Make sure the minimum font size of text in your story is large enough so that it can easily be read by everyone.
- Provide links in the agenda doc that are relevant to your story, such as, issues, epics, Figma files, FigJam boards, and recordings.
- When sharing directly from Figma files and issues, consider preparing an outline of the things you want to cover beforehand, filling in a few details of points you want to mention. Use this as a guide while you speak to make it more linear and easy to follow for the audience.
- Is your internet connection limited? Stop the Zoom camera feed or try these tips.
- Ask people to share their questions after a section of your story.
- It's normal to have anxiety when everyone is looking at you. Remember: We're here to support each other, not to judge each other.
- To increase collaboration and transparency after the meeting, consider linking to your recording from the related epics or issues.

For more tips, see the communication handbook page.

Hosting

Product Design Managers take turns hosting the UX Forum. Managers can use the UX Forum issue template to create a tracking issue for their hosting date. See how to use templates.

Recording the sessions

To limit the amount of post-UX Forum editing, please start and stop (not pause) the recording after each topic. This will ensure that each topic saves as an individual clip, which is much faster to edit than a 1-hour clip. Choose "record to computer" each time you record.

Sharing on GitLab Unfiltered

If you need to edit the videos at all, you can use a video editor like iMovie.

Otherwise:

1. Locate the video clips on your computer.
1. Name the videos with "UX Forum" + the title of the topic.
1. Check with speakers to make sure the video is appropriate for the public (no customer names).
1. Upload the videos to GitLab Unfiltered.
1. Add the videos to the UX Forum playlist, and the UX Team playlist.
1. Set the visibility to Public.
 1. NOTE: If the video content contains items that are unSAFE mark the video as Private.
1. Share the YouTube links in ux and product Slack channels with a brief description of each topic.
 1. Add the same update to the Engineering Week-In-Review document. The document is available in the description of our internal engineering-fyi Slack channel.

Helpful tips

- If you cannot find the videos in your Zoom account, reach out to the calendar event owner, as it probably uploaded to their account.
- In iMovie, you can upload a video straight to YouTube, as long as it's under 15 minutes long. But you still have to go to YouTube to select the playlist.
- You can pick a better thumbnail and edit other settings after uploading by going to the Uploads page and selecting your video.
- More about using YouTube.

Updating the schedule

After the sessions are over, update the UX Forum schedule.

SECTION: product/ux/ux-scorecards.md

Intro and goal

The UX Scorecard is a way for us to identify and score the usability of an experience in our product based on a set of heuristics. We use UX Scorecards to build empathy with users and to quickly spot opportunities for improvement.

UX Scorecards should be done on every main job and should be repeated on a regular cadence. Consider refreshing the UX Scorecard every 6 months, or after significant changes or improvements are made. This will allow you and your team to continuously monitor the progress in making experiences better for our users. If it has been a while since a scorecard has been run in your group you should plan on doing one soon to reestablish the cadence.

As UX practitioners, we must think strategically about fixing usability challenges within the GitLab product in order to give our users a quality experience. Creating a UX Scorecard with associated recommendations enables us to identify, scope, and track the effort of addressing usability concerns within a specific workflow. When it's complete, we have the information required to collaborate with Product Managers on grouping fixes into meaningful iterations and

prioritizing UX-related issues.

See all the completed scorecards below:

UX Scorecards

Category Maturity Scorecards

About the process

A UX Scorecard can be completed in 1-2 days. Some workflows might take a little longer but generally, if the process takes longer than this, then consider reducing the scope. The UX Scorecard process is meant to balance flexibility and consistency. There are several ways you can create your Scorecard, listed from lightest to heaviest. Select an appropriate approach based on the time you have, the priority of the workflow to users, or whether or not this is the first Scorecard for a job.

Scorecard approach

When doing a Scorecard, you have options:

Option A: conduct a heuristic evaluation

Review the current experience by doing a heuristic evaluation. This can be done in half a day, and it is a fast approach especially if you are working on a Scorecard for a task that has previously been scored.

During the evaluation strive to wear the hat of the persona(s) relevant to the job and while doing so try to see the UI from their perspective as if they were a new user. As you progress through your evaluation this will be easy to forget so it's recommended to put a reminder somewhere in your view, such as a post-it stuck on your monitor that says "You're a new user!"

Bear in mind that a Heuristic Evaluation is considered an expert evaluation, discount usability method. Where the "expert" in this context is a UX expert, not a user expert. It can therefore be thought of as a starting point for finding potential problems, not necessarily an endpoint. If you do find some areas of improvement then you might want your next step to be the Option B UX Scorecard outlined below as a way to validate things with real users.

Option B: perform a formative evaluation

You can do a formative evaluation by having internal or external users try to accomplish the job. The goal is to provide the participant context (the scenario) and listen and watch how they attempt to complete the job. What we learn may differ from participant to participant.

When doing a formative evaluation, do a "light" usability test to observe 3-5 internal or external users, as this provides valuable insights and removes subjectivity. This can be done in about a day, if the scenario you are evaluating is simple to set up in a test project. If your scenario is highly technical and requires complex customizations, plan ahead as it can take a couple of days just to set up an environment for the evaluation. Our preferred set up is an unmoderated study, but you may also moderate if that will result in a better evaluation.

In every case,

- define the job prior to conducting the Scorecard.
- use our heuristics when evaluating the experience.
- use the Grading Rubric to provide an overall measurement that becomes the benchmark score for the experience.
- observation of users is preferred over a purely heuristic evaluation as it will remove subjectivity.

This is a process intended to help inform the design process and maintain a high bar of quality.

How UX Scorecards relate to Category Maturity Scorecards

When to create a UX Scorecard:

- To quickly identify and prioritize usability issues and opportunities for improvement within an experience.

- When joining a new product area to become familiar with the workflows and to evaluate the experience from a fresh perspective.

- In conjunction with usability testing workflows.

- In order to evaluate the onboarding experience of your workflows.

When to create a Category Maturity Scorecard:

- When more rigorous testing is required within a category in relation to a set of business requirements and/or features that are needed for the category to move up. This is a summative process (not for identifying usability concerns) that allows us to gather metrics and data that will help us understand how changes to the product impact the user experience over time. We grade the maturity of our product using this process.

How to do a UX Scorecard

Note that every scorecard is not the same. Product Designers are welcome to adapt the steps to their needs as long as they are as objective as possible and the spirit and outcome remains the same.

Use the UX Scorecard Template in the GitLab Design project to create the issue and follow the instructions in the template. If you adapt the method, note that in the description.

If you'd like to view or edit the templates, you can find them here:

UX Scorecard

Heuristic Buddy Scorecard

Grading rubric

Overall Experience

Following each scenario that is evaluated, a set of three questions are asked to help determine the appropriate score, or grade, to attribute to the scorecard.

Question 1: Single Ease Question (SEQ)

·Overall, this scenario was·

- 5 - Extremely easy
- 4 - Easy
- 3 - Neither easy nor difficult
- 2 - Difficult
- 1 - Extremely difficult

Question 2: Satisfaction rating

·How would you rate the quality of this experience?·

- 5 - Extremely good
- 4 - Good
- 3 - Neither good nor bad
- 2 - Bad
- 1 - Extremely bad

Question 3: UMUX Lite, adjusted (Useful)

·You just experienced our implementation of <Scenario>. How would you agree or disagree with the following statement:

<Scenario> has the features I need for what I need to do in my own work.·

- 5 - Strongly agree
- 4 - Agree
- 3 - Neither agree nor disagree
- 2 - Disagree
- 1 - Strongly disagree

Calculating the score

To determine the overall score, use the UX Scorecard Calculations sheet (internal only).

Badge	Summary	Description
-----	-----	-----
	High Quality / Exceeds Expectations	Experience exceeds expectations and the user feels the experience is delightful. - Ease: Extremely easy - Satisfaction: Extremely good - Usefulness: Strongly agree
	Meets Expectations	Meets expectations but does not exceed user needs. The user is able to reach the goal and complete the job. - Ease: Easy - Satisfaction: Good - Usefulness: Agree
	Average	The user can complete the job but it does not exceed their needs and requires unnecessary steps. - Ease: Neither easy nor difficult - Satisfaction: Neither good nor

bad
- Usefulness: Neither agree nor disagree |
| | Poor | Experience is viewed as a poor experience and is difficult to complete.
- Ease:
Difficult
- Satisfaction: Bad
- Usefulness: Disagree |
| | Terrible | Too many users are unable to complete the job. Experience is viewed as
extremely bad and extremely difficult to complete.
- Ease: Extremely difficult
-
Satisfaction: Extremely bad
- Usefulness: Strongly disagree |
| | Unknown | This job has yet to be graded.
- Ease: Unknown
- Satisfaction:
Unknown
- Usefulness: Unknown |

Onboarding can refer to many different scenarios, and this can impact the experience:

- a SaaS or self-managed user
- a brand new GitLab admin setting up a SaaS or self-managed account/instance
- a brand new GitLab user joining a company or team
- a trial user
- an existing user of GitLab joining a company or team
- an existing user of GitLab trying a feature in a new stage or category
- a GitLab feature, a set of features, or a complete DevOps stage

For example, an existing user of GitLab joining a new team might require some help on unfamiliar features, or getting oriented to the groups and projects on the team. While a brand new user would require more help getting oriented with the application itself.

Steps to Evaluate Onboarding UX

- Think about the most important tasks/scenarios related to your JTBD.
- Identify the scenarios in which a user would do this task for the first time.
- Conduct a UX scorecard or usability test on those tasks/scenarios, incorporating each scenario.
- Focus your evaluation on the onboarding heuristics and rubric to rate the onboarding experience.
- Document the onboarding score and date (location TBD).

If you have a recent UX scorecard or a recent usability test with recordings, you can update these rather than starting over. For example, if you recently did a usability test on creating MRs, you can re-watch the sessions with the onboarding heuristics in mind to infer an initial score for the onboarding experience. However, it is highly recommended that at some point you intentionally evaluate the onboarding experience.

SECTION: product/ux/category-maturity/_index.md

As GitLab product maturity evolves, we must determine the capabilities that will move us to 'Best in Class' in the product categories we support. Our goal is to identify functionality and usability gaps as early as possible in the product development lifecycle using a lightweight and repeatable process that measures the maturity of our own capabilities against top competitors within the DevOps market.

How we assess category maturity

While there is flexibility in how groups approach this process, the recommended approach is:

1. Define the key Jobs to Be Done (JTBD) for a category.

- Starting here allows us to step back from our business and understand the objectives of the people we serve. It also gets us thinking more about the problems we're solving, rather than the features we're building. "Feature-itis" can become a trap when we assume that more features are always better or that our competitors have rigorously evaluated their own feature sets.

1. Perform a Competitor Comparison to document how we and our top competitor address the JTBD.

- The goal at this step is to use our own industry expertise, both in product-market fit and usability, to identify existing product gaps that we believe we should address. The Product Manager leads their entire stage group team plus their Marketing counterpart in running this analysis.

1. Determine where in the maturity cycle the gaps should be addressed.

- For example, what do we need to address to get to Competitive What can we save for Complete? We can document the path to the next maturity level in Epics and associated issues.

1. Conduct the Category Maturity Scorecard process

- When we think we've reached the next maturity level, we perform rigorous user research with users to validate whether we were right.

SECTION: product/ux/category-maturity/competitor-comparison.md

This handbook page outlines the competitor comparison process to use in conjunction with Category Maturity Scorecards.

Why it's important to look at competitors

Our existing Category Maturity Scorecard process focuses on usability from the perspective of our users. What that process lacks is a look into how we, as an application, compare in functionality and features against our key competitors with our Jobs to be Done (JTBD) in mind. In other words, adding a competitor comparison gives a more rigorous view into "here's the problem we're trying to solve" (JTBD) combined with "here's how it's being solved" (features/functionality) and "here's how pleasant it is to use" (Category Maturity Scorecard).

To address that gap, this competitor comparison piece is designed to fit within the existing Category Maturity Scorecard process as an add-on component and another data point to use when measuring category maturity.

In this new process, we're not conducting a direct feature comparison. Instead, we're focusing on the problems we're trying to solve (otherwise known as the JTBDs) and how they're being solved (otherwise known as features) within GitLab vs. our competitors. Ultimately, this will help us understand what 'feature complete' may look like for a given JTBD within the industry.

Fitting into the existing Category Maturity Scorecard process

You can conduct a competitor comparison at any time, but it's strongly recommended to conduct it early, prior to reaching Minim